



» Use Case Recycling by Extracting Business Rules

Statistics: ★★★★★ (776 Views) (3 [Comments](#))

Categories: [Use Cases](#), [Business Rules](#)

Introduction

Have you ever thought the following thought while describing business processes? "Hm, I have specified something very similar before. When was it? Ah, that other project I was working on last year. But that was a completely different topic!"

Welcome to reality. A decent percentage of the work business analysts (BAs) do is repetitive in nature. To abstract things, to move from fact to vision, and thereby improve reusability is useful for any business analyst striving for efficiency and effectiveness.

In this article I show how to extract business rules from use case descriptions. Business rules define the various subject matters, different businesses, or just differences in doing business. Use cases describe the interaction between user and system on a business level, i.e. with a result that is useful to the user. Therefore a use case from which business rules have been extracted provides a nice description of the WHAT of some part of a business process. Furthermore, it is quite robust to the ever-occurring changes in today's business, too. The extracted business rules, on the other hand, can be handled more easily, facilitating change in the software to be built. Ultimately, this approach leads to re-usable patterns of use case content.

I provide some real life examples and a detailed process description.

Defining the Goal

Just recently, I have again used a use case approach to a business process reengineering (BPR) problem. An aging application of the several 1,000 function point category, heavily 'enhanced' and somewhat maintained to death in 7 years of operation, had to be split up into distinct business modules. The aim was restructuring along natural business module borders, so that different applications (new and existing ones) can handle the business process more efficiently, more secure and within the right stakeholder's responsibilities. The task at hand was to write a requirement specification for some part of the new system of systems.

As almost always, documentation of the business processes had undergone aging as well. More specifically, it was incomplete, not up to date, and full of design decisions which did or did not make it to the actual application (who knows?). In short, the existing documentation was not trustable. The application itself also was suspect, with quite a number of defects and the ominous workarounds usually implemented to minimize the shortcomings of an aging design, caused by taking shortcuts in the maintenance phase.

So I decided to make use of the knowledge of a couple of subject matter experts (SME). These people seldom have any knowledge of business analysis methods (which is not their fault, nor their obligation, of course). In this case an approach that emphasizes scenario thinking is most likely to give useful results.

Here are the goals I had in mind when I made the decision: A requirements specification should arrive at several end states, specifically

- **Readable.** Comprehensive and comprehensible to the intended readership (stakeholders, analysts, architects, designers, developers, testers, maintenance people, they all should easily understand the written statements).
- **Clear.** Clear and specific enough to deduce a design, and tests.
- **No Design.** Free from unnecessary constraints concerning the design (although for system replacements, you can have quite a lot of necessary ones, consider the old interfaces to neighbouring systems).
- **Structuring.** Uncovering the borders between different business modules (this is one of the main points of any BPR approach)
- **Re-usable.** Re-usable across the system, or even various systems (re-use of development artefacts can boost project productivity quite bit, see [\[1\]](#))
- **Changeable.** Changeable during the life-time of the application (and perhaps beyond, as the need for the described project clearly shows).

Details and Examples

Early in the project we elicited functional requirements with system use cases, i.e. using a use case form (textual and

graphical) to model the required interaction between user and system (application). As in previous projects the hardest part of the work of the SMEs was letting go of the concrete implementation of the old application. This includes abstracting the interaction between user and system to a level of implementation independence. I like to call this step 'from fact to vision'. Note that from their point of view the old application's behaviour was the business process, although most common definitions of the term 'business process' have nothing to do with any software implementation (for instance, see [2] or [3]).

For example, stakeholders would describe a use case's flow of events like the one shown in Table 1:

Use Case Name: Place order
System: Restaurant Order System

Step	Label	Description
#1	Display options	The mobile device displays the options.
#2	Choose order option	The registered guest chooses „Order“ from the mobile device.
#3	Display menu	The mobile device displays this guest's order menu with all dishes, drinks and extras the guest may choose from. <i>Comment: never show the items we know the guest does not like.</i>
#4	Choose items	The guest chooses the dish and a drink by ticking the respective box next to the item on the order menu.
#5	Confirm order	The guest confirms the order by means of the mobile device's OK button.
#6	Display confirmation	The mobile device displays a confirmation message with all the details of the order.
#7	Check availability	The mobile device checks availability of the order items with the kitchen. <i>Comment: The system shall not allow orders for items that are not available.</i>
#7.1	Display error message	If one or more of the items chosen is not available, the mobile device displays a message with the details. Start over at step #3
#7.2a	Recalculate items	If all items are available, the mobile device subtracts 1 from the available items table for each item ordered.
#7.2b	Send order	The mobile device sends the order to the kitchen display.

Table 1: An example use case description's flow of events.

Note that I made up the example, in order to fulfil the nondisclosure agreement with my current employer. However, concerning the various phenomena of what is wrong with this use case, it is an example very close to reality. The example is based on an educational use case from the SOPHISTs [4].

Note that, by the selection and the sequence of the steps, unnecessary design is included in the requirements specification. Not only does this violate a rule, as implied by the goals in the previous section, it even is a bad design. In the sequence above the user will be prompted to order items which may or may not be available. Together with the ominous 'loop' in step #7.1 this makes a very frustrating user experience. Additionally, it gives away most of the pre-sumed advantage of such an IT based order system over the human waiter (or waitress, speaking of advantages). Instead, the availability check should be designed to happen before the menu options are displayed, so that unavailable items cannot be ordered in the first place. The use case description has to allow this design. There are more than just business rules related flaws in the description (like the various user interface specific requirements), but I guess you get the point. Why does things like this happen? Because the legacy application does things that way, and it actually is hard for most people to move from fact to vision, to put the ends foremost in mind, not the means.

The above description includes many different aspects of the respective part of the business process. The most prominent are

1. the possible flow(s) of events of the interaction of user and system,
2. the results the user wants to achieve by using the system, and
3. the rules that define the business itself.

The first two aspects have been covered at length elsewhere (for example in [5]), so let's look at the business rules aspect. A business rule is either

- A **Constraint** - describing mandated criteria such as "accessing the X-files requires Mulder's or Scully's privileges", "the applicant must not be a felon",
- A **Guideline** - describing a policy, guideline, standard, or regulation upon which the business operates, upon which business decisions are made or suggested such as "a retired postal code should not be re-activated for 1 month", "the applicant should not have been rejected previously",
- An **Action Enabler** - asserting some business structure, or controlling or influencing the behaviour of the

business, or rules that initiate or trigger behaviour, like "after 3 consecutive unsuccessful logins, ...", "when the customer's card has been rejected, it must be confiscated",

- A **Computation** - describes the embodiment of calculations, like "pay 1.5 times the wager for a two-card total of 21", "on the waiting list, move the customer up two positions" or
- An **Inference** - using "if...then" statements

Not every use case has one or more each, this is just a list of possible categories.

These are the business rules one can obviously extract from the above use case example.

Business Rule Number	Name	Description	What is it?*
#1	Guest Registration Status	Guests must be registered to gain access to the mobile device's functions.	Constraint
#2	Guest's Menu	Offered food and drink items mustn't be disliked items.	Constraint
#3	Availability	The system shall not take orders for items that are not available.	Guideline
#4	Reduce Stock	Any placed order must be subtracted from the list of available items of the respective dish, drink or extra.	Computation
#5	Possible Orders	Orders can include dishes, drinks and extras.	Constraint

Table 2: The business rules extractable from the example.

* You don't need to include this kind of information in your list. Shown here for clarity only.

Best practices on how to document business rules will follow below.

And this is what remains in the Use Case Flow of Events:

Step	Label	Description
#1	Choose Order option	The guest chooses the „order“ option.
#2	Display menu	The mobile device displays this guest's order menu.
#3	Choose items	The guest chooses dishes, drinks and extras.
#4	Inform kitchen	The mobile device informs the kitchen of the order.

Table 3: The flow of events freed from business rules.

At first glance, the description of the flow of events seems shorter, more business-oriented, and less laden with details. Hence, it's easier to read and understand. But let's have a closer look at the various advantages.

The sequence of steps in the use case is now much more universal, almost readily re-usable for similar purposes. In fact, the purpose is largely determined by the business rules in Table 2. Note that the description now has gained quite some independence from implementation specifics. The essential characteristic of the use case description comes out. This greatly widens the array of possible designs, and the "pre-check availability" design I suggested above meets the requirements neatly. The designer now has a better chance of not only fulfilling the business process specific requirements, but to also balance the design, in order to meet the set of all stakeholder requirements. Hence, he can engineer the application to actually be useful with respect to the stakeholder's goals (not 'just' their stated requirements).

In addition, keeping the business rules separate from the dynamic flow of the application opens up possibilities to implement a change-robust design. Imagine the business changes somewhat in the future, and Rule #1 is no longer a business rule. If the system architect makes an explicit decision** within the architecture or the design to implement customizable business rules, this kind of change would require very little resources to realize. It's also relatively safe to do the change in the software, as side effects can be minimized. By safe I mean that the programmer will inject no or only few new defects while touching the code, and maintainability does not decrease significantly by doing so.

** However, you might not design for this kind of flexibility unless you are sure it has a business value. This should not keep you from writing the requirements with business rules separated! Writing business rules separately may prod you towards a respective refactoring step.

How to Extract Business Rules

From the various times I used the extracting approach, and with the help of Scott Selhorst, author of the highly recommendable Tyner Blain weblog [6], I arrived at the following description of a process to extract business rules from use cases. You can use it as a checklist for handling existing use cases, or a guideline for BAs or SMEs who write use

cases. Of course, doing the latter is more efficient, but not always possible.

An always up-to-date version of the process can be found at Planet Project [7] as well, my wiki with dozens of checklists like this. I invite you to participate in the wiki and to improve and augment the material.

Process: Extracting Business Rules

Tag: process for extracting Business Rules from Pragmatic Use Cases; Version: 2009-03-10; Owner: Rolf.Goetz@gmx.de; Status: revised draft; Source: <http://planetproject.wikidot.com/improve-reusability-of-use-cases-by-extracting-business-rule>

Entry Conditions

E1: Use cases have been written (and validated by subject matter experts (SMEs)) *'Sometimes it's easier for SMEs to validate business rules when they see them already separated from the rest. Sometimes not.'*

E2: You are prepared to document the extracted business rules properly. *'Be it as a separate part of the use case template (simple) or in a list of business rules for all use cases (advanced, and far better for more than 5 use cases or so).'*

Steps and Notes

S1: Assume you have a use case with sections of some sort for a description, triggers, preconditions and normal / other flows. *'other written material that says something about the business could be just as good.'*

S2: In every section find statements that constrain the use case. Look for the keywords "must be", "must have", "make sure that", and also "always", "every", "never", "no", "all", "may", "can". *'careful here, make sure that the statement is true, i. e. validate it.'*

S3: In every section find statements that enable actions. Look for the keywords "when", "if", "as soon as", "in case", and time triggers.

S4: In every section find statements that describe computations. Look for factors and divisors, formulas in general. *'What else can be named a computation?'*

S5: In every section find statements that describe inferences: Look for "if ... then".

S6: In every section find statements that describe default values. *'A classical rule to be implemented as a parameter.'*

S7: Especially in the flow sections, look at every decision or branch and find the rules that govern the decision making in this particular spot.

S8: Search the Normal, Alternative and Exception Flow Parts for concrete values like key-words, number ranges, states. Their abstractions are the business rules. *Note: this also helps with finding the essential steps in a use case.'*

S9: For every statement found, see if you can extract it into a whole new sentence AND re-write the remaining part of the description by using a reference to that new sentence. If you can, you have probably found a business rule in that statement. Give it a name and or a number for reference purposes.

S10: Make sure your Business Rules can be clearly distinguished from the rest of the specification. *'By putting them in an extra chapter of your specification.'* If you're using a word processor consider using references, so that you only need to write each rule once, and display it many times.

S11: Consider writing an explicit requirement that requires some robustness concerning the change of these rules. *'you will have to predict in which way and to what extent you expect the rules to change'*

Exit Conditions

One may exit the process, if you are sure there are no more Business Rules hiding. You can do this by using the above definition of Business Rule as a checklist for sifting through all use cases.

Note: With some practice you can use steps S2 through S8 to form questions for interview purposes.

Arriving at Content Patterns

If you follow this structuring route of thinking to it's apparent end, you arrive at Martin Langland's Use-Case *Content* Patterns [8], which are patterns "of flows, steps and other features that addresses a commonly-encountered Use-Case modelling problem, and that can be applied again and again to specific instances [of use case specifications]." Addition in brackets is mine. Martin's patterns have a very universal, yet comprehensible quality. Context is only defined by the business perspective you use them for, as represented largely by business rules.

In this case the SELECTOR pattern would be appropriate. It goes along these lines (slightly changed from the original source to match wording, and annotated for the above example):

1. The Actor defines the search values, against a set of criteria. *'see the business rules. Criteria could be Guest*

Registration Status rule #1), Guest's Menu (rule #2), Availability (rule #3), and Possible Orders (rule #5).

2. The System finds object-instances that meet the values set. *'this defines the set of dishes, drinks and extras to be presented to the guest'*
3. The System displays the found object-instances, displaying a pre-defined set of properties for each. *'properties could be the name, the ingredients, the price, and some 'ordered before' status'*
4. The Actor selects one or more of the displayed objects, and indicates that the selection is complete. *'I want this and that, please bring it. Core of the use case.'*
5. The System returns control to the client use case, returning the collection of selected objects. *'here, the client would be responsible for transferring the order to the kitchen'*

There are several interesting things to note on this kind of description. First, it is completely free of business context. Even the actors are abstract. Hence, it is universal. Second, the invoking action of the use case in Table 3 (step #1) is no longer part of the flow of events. It could now be used as the use case trigger. Third, the flow does not mandate some weird, unfriendly design. Though it mandates a design.

If you don't have use cases or other material from which business rules could be extracted, I strongly suggest using content patterns for writing system use cases from the beginning. Be careful to exclude unwanted design. Supplement the specification with collections of the business rules you find along the way.

In the project I introduced before we did not use the patterns, though. The reasons were manifold, but completely political in nature. I assume we could have reduced the effort to arrive at a decent set of use cases by at least 30%, more likely by 50% or more, if we had used content patterns in the first place. However, extracting business rules from the pragmatic descriptions was part of the process. It proved to help a great deal to arrive at the goals I listed in section 2, Defining the Goal.

Writing Business Rules

By following the above process, you arrive at a list of business rules like the one in Table 2. There are two characteristics a written business rule should have, in order to be useful for business analysis purposes. A business rule should be

- **atomic**, that is, it cannot be further decomposed.
- **process-independent**, so that it applies to the application at all times (not just at the very instance within the flow of events of the use case).

I find a complete list of business rules with the rules from each use case all gathered in one place is much easier to maintain. The other approach, keeping the rules with the use case descriptions is prone to error, because of redundancy (or more disastrous: very close redundancy), incompleteness, and contradiction. The list of business rules can be easily validated by SMEs. Just read through and tick off true statements, true within the area of concern.

The business rules need to be referenced, so give each a unique tag or ID. For an example, see Table 2. Ideally, an ID sticks with the respective rule and will not be reused in case a rule gets deleted. This helps prevent misunderstandings.

Conclusion

In this article I showed the advantages of separating use case descriptions from business rules. You don't have to edit the use case if some detail of the business changes. You can re-use the use case descriptions across the application or even across several applications, and ultimately arrive at universal use case content patterns. Your use cases will make a great step towards implementation independence, towards 'no design'. And, with only one additional conscious decision in your design, you can achieve a new level of change-robustness within your application.

Extracting can be done by the process provided. You can also use the process description as a checklist for separating business rules and use cases while writing use cases. The article then connected the extraction approach to Martin Langland's use case content patterns. They lead to truly universal use cases and are an excellent option to begin with. Finally, several best practices concerning documentation of business rules were given.

Enjoy use case recycling!

Sources

- [1] C. Jones, Applied Software Measurement, McGraw-Hill Osborne Media; 3 edition (April 11, 2008), ISBN 978-0071502443
- [2] Wikipedia, [Business rule](#) (nice best practices!) (as of Jan. 30, 2009, 08:04 GMT).
- [3] The IIBA, [The Business Analysis Body of Knowledge, Version 1.6](#)
- [4] SOPHIST, www.sophist.de
- [5] A. Cockburn, Writing Effective Use Cases, Addison-Wesley Professional (October 15, 2000), ISBN 978-0201702255
- [6] S. Selhorst, [Business Rules Hidden in Use Cases](#) (as of March 10, 2009, 09:52 GMT)
- [7] R. Götz, [Improving Reusability of Use Cases by Extracting Business Rules](#) (as of March 10, 2009, 17:00 GMT)
- [8] Use [Case Content Patterns](#), Martin Langlands (as of March 1, 2009, 20:14 GMT)

Author: "Rolf Götz (born 1971) is business analyst, author and consultant. He regularly publishes articles and blog posts on various IT and soft skill topics, including business analysis, engineering and project management. Mr. Götz has been with the Deutsche Post AG since 2005. There, he supports about 10 larger system developments from the Operations domain. Typical roles he would play are portfolio manager, requirements engineer, process consultant and quality manager. Before that, he was into requirements engineering of IT systems in various other business domains, including air traffic control, avionics, weapon systems, medical systems, health insurance, and logistics. Rolf writes the Clear Conceptual Thinking blog (ClearConceptualThinking.net) and maintains the Planet-Project wiki (PlanetProject.wikidot.com), with close to 100 useful checklists on requirements, systems and projects. The opinions expressed here represent my own and not those of my employer.

Comments

(Aaaargh!! When you click on the rating your comments are wiped out.)

So as I was saying ..

- Why do we care about the 'type' of business rule?

- Where possible include the business rule in the use case.

#1 is a precondition to the use case.

#2 and 3 Add to step 2 that displayed items are liked and available.

#4 is describing an implementation of a way of keeping track of available items (I assume).

#5 is a definition for the project glossary.

Since every requirement has an associated cost, my recommendation is to look for the business rules at every step in the use case and where possible add them to the use case steps, not to track as separate requirements.

Nice article, thanks,

Leslie.

#1Guests must be registered to gain access to the mobile device's functions.

#2Offered food and drink items mustn't be disliked items.

#3The system shall not take orders for items that are not available.

#4Any placed order must be subtracted from the list of available items of the respective dish, drink or extra.

#5Orders can include dishes, drinks and extras.

posted @ Monday, March 30, 2009 12:07 PM by baldrick

Comments from the following blog entry: [Keeping business rules out of your use cases with decisions](http://www.ebizq.net/blogs/decision_management/2009/04/keeping_business_rules_out_of.php), located at:
http://www.ebizq.net/blogs/decision_management/2009/04/keeping_business_rules_out_of.php

posted @ Wednesday, April 01, 2009 10:45 AM by James Taylor's Decision Management

Comments from the following blog entry: <http://jtonedm.com/2009/04/02/keeping-business-rules-out-of-your-use-cases-with-decisions/>

posted @ Thursday, April 02, 2009 1:46 PM
